TerraMind on SmallMinesDS: Experiment Report

Oct 20, 2026 · Asım Emre Arslan

Summary

Under the plan's fixed 10-epoch protocol, the frozen pretrained TerraMind v1 tiny encoder beats the same architecture trained from scratch from N = 20 training images upward (mine-class IoU 0.591 vs 0.564 at N = 250), and D4 geometric augmentation helps mainly at N = 10–20. Below N = 10 the protocol trains for only 10 optimiser steps, so the main experiment cannot compare the encoders there; a supplementary run with an equal 320-step budget closes that gap.

- **N ⩽ 5 is uninformative in the plan's protocol.** Every run ends in a constant prediction (all mine or all background) and the four conditions are identical to three decimals (IoU 0.047 ± 0.038).
- **Pretrained frozen encoder leads from N = 20**, and the margin grows with N: +0.018 IoU at N = 50, +0.016 at N = 100, +0.027 at N = 250 (without augmentation), with lower seed variance.
- **Augmentation is a small-data tool.** Main protocol: +0.03 IoU at N = 10 and +0.06 at N = 20 for the frozen encoder (5/5 seeds at N = 20), ≈ 0 from N = 50. With an equal 320-step budget: +0.11 to +0.14 IoU at N = 3–10 for both encoders (5/5 seeds), fading to 0 by N = 100.
- **With equal steps every budget learns**: IoU 0.14–0.21 from a single training image, 0.54–0.60 from 20. The frozen encoder is ahead from N = 20 (+0.05 at N = 250); the trainable random encoder is slightly ahead at N ⩽ 10, within one standard deviation.
- **Fully reproducible.** All 480 runs (main + two supplementary grids) are bit-for-bit repeatable from the public repository; total compute 13 + 14 + 62 minutes on one RTX 3080 Laptop GPU.

Research question and design

The plan asks how training-set size and geometric augmentation affect binary mine segmentation when a frozen pretrained TerraMind v1 tiny encoder is compared with the same encoder architecture trained from scratch. The design is a 2 × 2 of encoder strategy and augmentation, crossed with eight training budgets and five seeds: 160 runs.

Condition	Encoder initialisation	Encoder training	Augmentation
A	TerraMind v1 tiny, pretrained	frozen	none
B	TerraMind v1 tiny, pretrained	frozen	random D4 symmetry
C	random	trainable	none
D	random	trainable	random D4 symmetry

- Budgets N ∈ {1, 3, 5, 10, 20, 50, 100, 250} unique training images; subsets nested within a seed and identical across the four conditions.
- Protocol: exactly 10 epochs, evaluate the epoch-10 weights; AdamW, constant learning rate 1e-4, weight decay 0.01, batch size min(8, N); unweighted 2-class pixel cross-entropy; reshuffle every epoch, final partial batch kept; no early stopping, validation set or checkpoint selection.
- Augmentation: one of the eight square symmetries (rotations by 0/90/180/270°, each with or without a horizontal reflection), drawn uniformly per image presentation and applied identically to S1, S2 and the mask; test inputs never augmented; N unchanged.
- Metrics: mine-class IoU, F1, precision and recall from argmax predictions and confusion counts pooled over the fixed 644-image test set; zero-denominator metrics reported as NA; curves of mean ± std over seeds against N; paired augmentation gains within each encoder strategy.
- Caveat from the plan itself: the encoder comparison changes initialisation and trainability at once, and a fixed epoch budget gives more optimiser steps as N grows (10 steps at N ⩽ 8, 320 at N = 250).

Data and preprocessing

SmallMinesDS 2022 holds 2,135 patches of 128 × 128 px over five districts of south-western Ghana, split by the dataset authors into 1,491 training and 644 test patches. Masks are binary (1 = mine); 7.6 % of all pixels and 7.8 % of test pixels are mine, and 511 training patches (34 %) contain no mine pixels at all. The pinned dataset revision is downloaded and checksum-verified by a script; every CSV row was matched to its image and mask file, and the CSV mine percentages agree with the masks to 0.005 %.

Inputs are the ten Sentinel-2 L2A bands B2, B3, B4, B5, B6, B7, B8, B8A, B11, B12 and the two Sentinel-1 RTC bands VV, VH; the DEM band is dropped as the plan requires. Sentinel-1 is stored linear in the GeoTIFFs and converted to dB (10·log10) because TerraMind was

pretrained on dB values. Both modalities are standardised with TerraMind's own pretraining mean and standard deviation, in every condition, so the four conditions see identical inputs.

The plan left several choices open; each was decided before any run and is recorded in the repository README.

Open point in the plan	Decision	Reason
Input size	Native 128 × 128, no resizing	Keeps the data as published; TerraMind's fixed sin-cos position table for 128 px is exactly the top-left 8 × 8 crop of its 224 px table, so pretrained weights load unchanged
Normalisation	TerraMind pretraining statistics for all conditions	Only the encoder should differ between conditions
Sentinel-1 flavour	S1RTC modality, values in dB	Dataset radar is RTC and speckle-filtered; TerraMind expects dB
S1 + S2 fusion	TerraTorch: joint token sequence in the encoder, per-layer outputs averaged (merge_method = mean)	Library default; keeps the decoder identical and small
Shallow decoder	4-block UNet decoder on a learned 4-level pyramid (details below)	Chosen by the project owner; identical in every condition
Subsets at small N	First 10 images of each seed's ordering drawn from the 140 training patches with 25–75 % mine pixels; the rest in random order	Avoids N = 1 subsets with no mine pixels (34 % of patches); keeps budgets nested; N ⩽ 10 are all such images and larger budgets contain at least those 10
Seed	One seed fixes subset, decoder initialisation and per-epoch batch order; augmentation draws from a separate seeded stream	Makes the four conditions of a seed a paired comparison

The subset rule departs from the plan's pure random ordering and makes small-N training sets far more mine-heavy (25–75 % of pixels) than the test set (7.8 %); precision and recall at N ⩽ 20 should be read with that in mind. The exact patch ids of every (seed, N) subset are stored in `results/main/subsets.json`.

Model

The model is built with TerraTorch 1.2.13 and is identical in every condition; only the encoder's weights and trainability change.

- **Encoder**: `terramind_v1_tiny` (12 layers, width 192, patch 16). S2L2A (10 bands) and S1RTC (2 bands) are embedded separately, processed as one joint token sequence, and the two modalities' outputs are averaged per layer. At 128 px each modality yields 8 × 8 tokens. In the pretrained conditions the weights come from TerraMind-1.0-tiny; all 152 weight tensors load unchanged at 128 px, and the 10-band Sentinel-2 patch embedding is the matching slice of the pretrained 12-band one.
- **Neck**: outputs of encoder layers 2, 5, 8 and 11 are reshaped to 8 × 8 maps and turned into a 4-level pyramid (32, 16, 8, 4 px) by TerraTorch's learned interpolation neck.
- **Decoder and head**: a 4-block UNet decoder (segmentation_models_pytorch) with 128/64/32/16 channels up to 64 × 64, a 1 × 1 convolution to two classes, bilinear × 2 to 128 × 128. Normalisation layers are BatchNorm2d (library default) in the main experiment; supplementary experiment A replaces them with GroupNorm.

Part	Parameters	Trainable in frozen-encoder conditions
Encoder (TerraMind v1 tiny)	5.92 M	no
Neck + decoder + head	0.97 M	yes
Total	6.90 M	0.97 M of 6.90 M

The decoder side is re-initialised from a seed-derived generator after the encoder is built, so for a given seed its initial weights are the same in all four conditions.

Training protocol and reproducibility

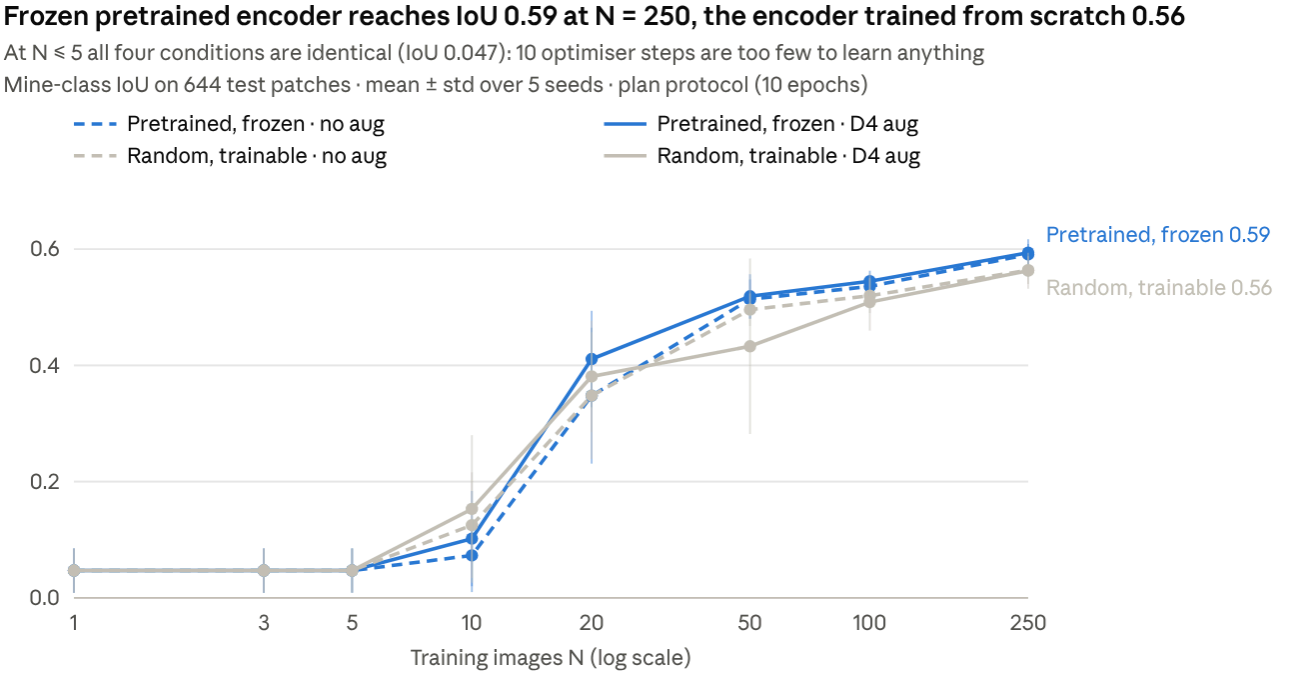
Every run follows the plan's protocol exactly: 10 epochs, AdamW (lr 1e-4 constant, weight decay 0.01), batch size min(8, N), unweighted two-class pixel cross-entropy, a fresh shuffle each epoch with the final partial batch kept, evaluation of the epoch-10 weights on the full test set, no early stopping or model selection. The number of optimiser steps therefore grows with N: 10 steps for $N \leqslant 5$, 20 at N = 10, 30 at N = 20, 70 at N = 50, 130 at N = 100 and 320 at N = 250.

Within a seed the four conditions are paired: they train on the same images, start from the same decoder weights and see the same batch order. Augmentation draws come from their own seeded generator, so switching augmentation on does not change which images share a batch. The random-init encoder is also initialised from the seed.

Runs are bit-for-bit reproducible on the same machine and software stack: cuDNN deterministic mode, `CUBLAS_WORKSPACE_CONFIG=:4096:8`, `torch.use_deterministic_algorithms(True)`, the math attention kernel instead of the non-deterministic flash kernels, and a cross-entropy written without the non-deterministic `nll_loss2d` CUDA kernel. Each configuration was repeated three times in a test and produced identical confusion counts and loss values; the refactored training loop used for the supplementary runs reproduces the stored main-experiment rows exactly.

Results of the main experiment

The frozen pretrained encoder is the better strategy from N = 20 upward, augmentation pays off at N = 10–20, and the budgets N ⩽ 5 produce no learning at all under this protocol. All 160 runs completed in 13 minutes.



results/main/summary.csv · 160 runs: 8 budgets × 4 conditions × 5 seeds

- **N ⩽ 5: nothing is learned.** Ten steps at lr 1e-4 leave every run at a constant output. Three seeds predict mine everywhere (recall 1.0, precision 0.078 = the test base rate); two predict background everywhere (IoU 0, precision NA). Which of the two happens is set by the shared decoder initialisation, so the four conditions coincide.
- **From N = 20 the frozen pretrained encoder is ahead**: +0.018 IoU at N = 50, +0.016 at N = 100 and +0.027 at N = 250 without augmentation (0.591 ± 0.018 vs 0.564 ± 0.024), with the smaller seed spread. F1 follows the same order (0.743 vs 0.721 at N = 250).

- **Augmentation.** Paired gain for the frozen encoder: +0.029 ± 0.028 at N = 10 (4/5 seeds), +0.063 ± 0.050 at N = 20 (5/5), then +0.005, +0.009 and +0.004 at N = 50, 100, 250. For the random encoder: +0.028 and +0.033 at N = 10–20, −0.063 ± 0.127 at N = 50 (one outlying seed; +0.001 ± 0.007 without it), −0.011 and −0.001 at N = 100 and 250.
- **Precision and recall.** At N ⩽ 20 recall is high and precision low in every condition, as expected from the mine-heavy small subsets. At N = 250 the frozen encoder reaches precision 0.735 / recall 0.756 without augmentation, the random encoder 0.685 / 0.766.
- **One outlier.** Random encoder, D4 aug, N = 50, seed 3: IoU 0.133 against 0.450 for its unaugmented twin, with near-identical training loss (0.602 vs 0.593); it marks 55 % of test pixels as mine. It is kept as-is (the plan has no checkpoint selection); its cause is examined in the supplementary experiments below.

Full tables for all four metrics are in `results/main/summary.md`; the per-run rows, including confusion counts and per-epoch training loss, in `results/main/runs.csv`.

Supplementary experiments

The main experiment exposes two limits that are not about the encoders: the fixed epoch budget (10 steps at N ⩽ 5) and the decoder's BatchNorm layers at batch sizes of 1–8. Two single-factor variants isolate them. Both are full 160-run grids with the same data, subsets, seeds, model, optimiser, metrics and scripts; they are reported separately from the plan-faithful main results.

- **A — GroupNorm decoder** (`results/groupnorm/`, 14 min): the nine BatchNorm2d layers on the decoder side are replaced by GroupNorm with 8 groups; the 10-epoch protocol is unchanged; the parameter count is identical.
- **B — equal optimisation budget** (`results/steps320/`, 62 min): every N trains for exactly 320 optimiser steps, the number the plan gives N = 250, with the GroupNorm decoder. At N = 250 this is the same protocol as A, and the two produce identical confusion counts.

A: the normalisation layer

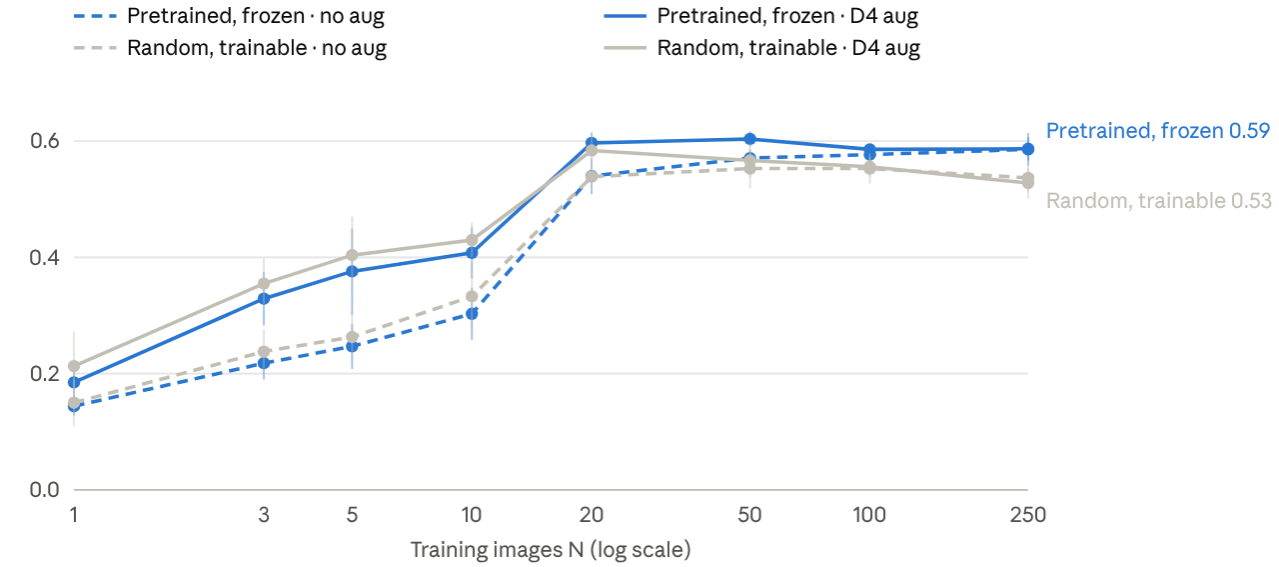
With GroupNorm no run is constant any more: at N ⩽ 5 IoU lies between 0.09 and 0.17 in every condition and no metric is NA. After only 10 updates BatchNorm's running statistics are still close to their initial values, so the network behaves differently in evaluation mode than during training; GroupNorm carries no such state. Under the 10-epoch budget GroupNorm converges more slowly at N = 20–100 (0.450 vs 0.514 IoU at N = 50 for the frozen encoder without augmentation) and matches BatchNorm at N = 250 (0.586 vs 0.591). The N = 50 seed-3 outlier persists (0.116 vs 0.284 for its unaugmented twin), so it is not a normalisation artefact.

B: equal steps for every budget

With 320 steps for every N, one image gives IoU 0.14–0.21 and 20 images 0.54–0.60

Frozen pretrained encoder leads from N = 20 (+0.05 at N = 250); the trainable random encoder is slightly ahead at N ⩽ 10

Mine-class IoU on 644 test patches · mean ± std over 5 seeds · 320 optimiser steps per run, GroupNorm decoder



results/steps320/summary.csv · 160 runs: 8 budgets × 4 conditions × 5 seeds

With 320 steps the frozen pretrained encoder is ahead from N = 20 and the margin grows with N: +0.001, +0.018, +0.024 and +0.049 IoU at N = 20, 50, 100 and 250 without augmentation. At N ⩽ 10 the trainable random encoder is ahead by 0.006–0.030, within one standard deviation. The frozen encoder saturates early: with augmentation, N = 20 (0.597 ± 0.018) already matches N = 250 (0.587 ± 0.020). The seed-3 outlier disappears (0.530 vs 0.538), which confirms slow convergence as its cause.

D4 augmentation adds +0.11 to +0.14 IoU at N = 3–10 with equal steps, and nothing from N = 100

Paired gain = augmented − unaugmented run with the same seed, subset and decoder init · whiskers = ± std over 5 seeds

Under the plan protocol (10 epochs) the gain appears only at N = 10–20, where the frozen encoder gains +0.03 and +0.06



results/steps320/aug_gain.csv and results/main/aug_gain.csv · paired differences, 5 seeds per cell

Under equal steps augmentation is a consistent small-data gain: +0.11 to +0.14 IoU at N = 3–10 for both encoders (5/5 seeds), +0.05 at N = 20, +0.03 for the frozen encoder at N = 50 (5/5 seeds), and about zero from N = 100. No mean gain is below −0.01. Full tables for A and B are in `results/variants.md`, `results/groupnorm/summary.md` and `results/steps320/summary.md`.

Discussion

The three grids agree on the ordering of strategies wherever the optimisation budget is sufficient; they disagree only where the protocol itself prevents learning.

- **Data efficiency.** The frozen pretrained encoder reaches most of its final quality with 20–50 images: in B, IoU 0.540 at N = 20, 0.571 at N = 50 and 0.586 at N = 250. The trainable random encoder needs the full 250 images to approach it and still trails by 0.03–0.05. Beyond N ≈ 50 extra data adds little under either protocol, which suggests the frozen-encoder + shallow-decoder combination has a ceiling near IoU 0.59–0.60 on this test set.
- **Augmentation** matters where the model sees each image many times. With 320 steps on ⩽ 10 images (32–320 epochs) D4 symmetries add +0.10 to +0.14 IoU; with the plan's 10 epochs each image is seen 10 times regardless of N, and the gain is confined to N = 10–20, where steps suffice to learn but data is scarce. From N = 100 the gain is within ±0.01 in every protocol.
- **Encoders.** At N ⩽ 10 with equal steps the trainable encoder can adapt its features to the few images, while the frozen generic features must be read through an 8 × 8 token grid by a shallow decoder; its advantage is small (0.006–0.030) and within one standard deviation. From N = 20 pretrained features generalise better and the gap widens to +0.05 at N = 250. Because initialisation and trainability change together, the experiment does not say which of the two drives the difference.
- **Protocol.** A fixed epoch count couples the optimisation budget to N (10 to 320 steps). Conclusions at N ⩽ 10 under the plan therefore describe the budget, not the data; the equal-step protocol separates the two.
- **Variance.** At N ⩽ 20 the spread across seeds (0.06–0.15 IoU) exceeds the differences between conditions; at N ⩾ 50 it falls to 0.01–0.03 and the 0.02–0.05 differences between encoders are consistent across budgets and protocols.

Limitations and deviations from the plan

Everything below is stated in the repository README as well; none of it changes the ordering of conditions at N ⩾ 20, but each affects how far the numbers generalise.

- **Confounded encoder comparison.** Initialisation and trainability differ at once, as the plan itself notes. A fifth condition, pretrained and trainable, is needed to separate them and was not run.
- **Subset rule.** The first 10 images of every seed are drawn from patches with 25–75 % mine pixels, not at random. Small-N training sets are therefore far more mine-heavy than the 7.8 % test base rate, which depresses precision and inflates recall at N ⩽ 20. The four conditions share the same subsets, so comparisons between them are unaffected.
- **Fixed 10-epoch budget.** Budgets N ⩽ 5 receive 10 optimiser steps and show no learning. The equal-step reading (B) departs from the plan and is reported separately.
- **Decoder choices are ours.** The plan says only “shallow decoder”; the UNet depth, channel widths and BatchNorm layers were chosen here. Supplementary A shows the normalisation layer alone moves IoU by up to 0.06 at N = 50 under the 10-epoch budget, although the ordering of conditions at N ⩾ 20 is the same in all three grids.
- **Resolution.** Images are used at their native 128 px instead of TerraMind's 224 px pretraining resolution. The positional table crop is exact, but the encoder sees 64 tokens per modality rather than 196, and feature quality at this size was not validated separately.
- **Five seeds.** Differences smaller than one standard deviation (all encoder differences at N ⩽ 10, most augmentation effects at N ⩾ 100) should not be interpreted.
- **One dataset, one year, one region.** Results describe SmallMinesDS 2022 in south-western Ghana with its published split. There is no validation set by design; the test set was used once per run and never for any selection.
- **One outlying run** (random encoder, D4 aug, N = 50, seed 3) is included as required by the protocol and is the main driver of the negative mean gains at N = 50 in the main grid and in A.
- **Reproducibility** is bit-exact on the machine and software stack recorded in `results/*/environment.json`; other GPUs or driver versions may differ in the last digits.

Recommendations and next steps

1. **Report the main grid as the plan’s result and read N ⩽ 10 through experiment B.** State the subset rule, the 10-step budget at small N and the BatchNorm effect wherever the small-N numbers are shown.
2. **Add a pretrained, trainable condition** (fine-tuning) with the same seeds, subsets and protocol. It separates initialisation from trainability and costs about 15 minutes for 80 runs with the existing scripts.

3. **For any follow-up on small budgets, fix the number of optimiser steps rather than epochs, and use GroupNorm in the decoder.** Both are already implemented (`--steps`, `--norm groupnorm`).
4. **Run a purely random subset variant** as a sensitivity check on the mine-fraction rule; expect higher seed variance and, in roughly a third of seeds, N = 1 subsets without any mine pixels.
5. **Check the resolution choice** by repeating the frozen-encoder conditions at 224 px (bilinear upsampling); if IoU moves by more than the seed spread, the 128 px numbers should be labelled as such.
6. Optional: the 2016 split of the same dataset allows a temporal transfer test of the N = 250 models at no training cost.

Reproducibility appendix

Code, results and figures: github.com/asimarslan11/SmallMinesDS-Experiment-Project (results as of commit `2d29c5b`, 20 October 2026). The dataset is not in the repository; the download script fetches revision `0cb168b8` of SmallMinesDS from Hugging Face and verifies its checksum, and TerraMind-1.0-tiny weights are downloaded by TerraTorch on first use.

```
python -m venv .venv && .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python scripts\check_gpu.py
python scripts\download_data.py
python scripts\build_cache.py
python scripts\run_experiments.py                                    # main
grid, 160 runs
python scripts\run_experiments.py --norm groupnorm --out
results/groupnorm/runs.csv            # A
python scripts\run_experiments.py --norm groupnorm --steps 320 --out
results/steps320/runs.csv  # B
python scripts\analyze.py [--runs results/<variant>/runs.csv]
python scripts\compare_variants.py
```

Item	Value
Hardware	NVIDIA GeForce RTX 3080 Laptop GPU, 8 GB; Windows 11
Software	Python 3.12.0, PyTorch 2.11.0+cu128, CUDA 12.8, cuDNN 9.19, TerraTorch 1.2.13, segmentation_models_pytorch 0.5.0, rasterio 1.5.2

Item	Value
Compute	main grid 799 s (13 min); A 853 s (14 min); B 3,728 s (62 min); 480 runs in total
Per-run record	seed, N, condition, confusion counts, IoU/F1/precision/recall, steps, per-epoch training loss, subset mine fraction, wall time, parameter counts
Subsets	`results/main/subsets.json`: patch ids of every (seed, N) training subset, identical for A and B
Summary files	`results/<variant>/summary.md`, `summary.csv`, `aug_gain.csv`, `figures/`; `results/variants.md` and `results/figures/variants.png` across variants
Determinism	cuDNN deterministic, `CUBLAS_WORKSPACE_CONFIG=:4096:8`, `torch.use_deterministic_algorithms(True)`, math attention kernel, deterministic cross-entropy; repeated runs identical in confusion counts and loss

Sources: SmallMinesDS dataset card, TerraMind 1.0 tiny model card, TerraTorch, and the project plan document *TerraMind v1 Tiny on SmallMinesDS — Experiment plan* (SmallMinesDS_TerraMind_Project_v1.docx).